MATERIA

DESARROLLO DE VIDEOJUEGOS (INF761)

UNIDAD 3

Físicas, Animación y Audio

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

3.1 Sistema de físicas de Unity
Rigidbody, Colliders, triggers, capas, joints y Raycasting

3.2 Animación
Animator Controller, estados, transiciones, Blend Trees, IK básico

3.3 Sistema de partículas (VFX)
Particle System, VFX Graph, impactos, explosiones y clima

3.4 Audio
AudioSource, AudioMixer, música adaptativa y sonido espacial 3D

3.5 Iluminación y post-procesado
Lightmapping, URP/HDRP, Volume profiles y shaders introductorios

3.6 Prototipo 2: Plataformero 2D/3D
Físicas, animaciones, VFX, audio, polish visual y Profiler

COMPETENCIA DE LA UNIDAD

Integrar sistemas de físicas, animación, efectos visuales y audio para crear prototipos de videojuegos con retroalimentación sensorial completa y pulido visual.

3. Físicas, Animación y Audio

3.1 Sistema de físicas de Unity: Rigidbody, Colliders, triggers, capas, joints y Raycasting

El sistema de físicas de Unity está basado en PhysX (3D) y Box2D (2D). Simula cuerpos rígidos, colisiones, fuerzas y restricciones de movimiento. Comprender la diferencia entre los componentes físicos y cuándo usarlos es esencial para construir gameplay responsivo y realista.

> **CONTINUIDAD DESDE UNIDAD 1**
> En U1.2 se introdujo Rigidbody2D como componente nativo. Esta sección profundiza en ambas variantes (2D y 3D), sus propiedades avanzadas, modos de detección de colisión y buenas prácticas de uso con FixedUpdate().

Rigidbody y Rigidbody2D

El componente Rigidbody transfiere el control del movimiento de un objeto al motor de físicas. A partir de ese momento, el objeto responde a gravedad, fuerzas e impulsos en lugar de moverse por Transform directamente.

Propiedad	Descripción	Valor típico
Mass	Masa del cuerpo en kg. Afecta fuerzas e impulsos	1–10 kg
Drag	Resistencia al movimiento lineal (fricción del aire)	0 = sin resistencia
Angular Drag	Resistencia a la rotación	0.05 por defecto
Gravity Scale	Multiplicador de gravedad (solo 2D)	1 = normal, 0 = sin gravedad
Collision Detection	Discrete, Continuous, ContinuousDynamic	Continuous para objetos rápidos
Constraints	Bloquear posición o rotación en ejes específicos	Freeze Rotation Z en 2D
Body Type (2D)	Dynamic, Kinematic, Static	Dynamic para personajes

```
// Siempre mover Rigidbodies en FixedUpdate, nunca en Update
void FixedUpdate()
{
    // Mover con velocidad (no usar transform.position con Rigidbody)
    rb.linearVelocity = new Vector2(moveInput.x * speed, rb.linearVelocity.y);

    // Aplicar fuerza puntual (impulso de salto)
    if (jumpPressed)
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);

    // Rigidbody Kinematic: mover sin físicas pero conservando colisiones
    rb.MovePosition(rb.position + velocity * Time.fixedDeltaTime);
}
```

Colliders

Los Colliders definen la forma geométrica usada para detectar colisiones. No requieren un Rigidbody para existir (Static Collider), pero la detección de colisiones entre dos Static Colliders no genera eventos. Siempre que un objeto deba recibir eventos de colisión, debe tener un Rigidbody.

Collider	Forma	Uso ideal
BoxCollider2D / Box	Rectángulo / Caja	Plataformas, cajas, objetos cuadrados
CircleCollider2D / Sphere	Círculo / Esfera	Personajes, balas, objetos redondos
CapsuleCollider2D	Cápsula (rect + semicírculos)	Personajes humanoides 2D
PolygonCollider2D	Polígono arbitrario	Terrenos irregulares, sprites complejos
EdgeCollider2D	Línea poligonal abierta	Suelos con pendiente, bordes de nivel
CompositeCollider2D	Unión de múltiples colliders	Tilemaps, entornos complejos
MeshCollider (3D)	Malla 3D exacta	Terrenos 3D, objetos irregulares

Triggers

Un Trigger es un Collider con la opción 'Is Trigger' activada. No genera respuesta física (no empuja objetos), pero sí genera eventos de entrada y salida que pueden usarse para zonas de daño, checkpoints, áreas de activación de diálogos y puertas.

```
// Eventos de colisión física (requiere Rigidbody en al menos un objeto)
void OnCollisionEnter2D(Collision2D col)  { /* primer contacto  */ }
void OnCollisionStay2D(Collision2D col)   { /* contacto continuo */ }
void OnCollisionExit2D(Collision2D col)   { /* fin del contacto  */ }

// Eventos de trigger (Is Trigger = true en el Collider)
void OnTriggerEnter2D(Collider2D other)   { /* objeto entró al área */ }
void OnTriggerStay2D(Collider2D other)    { /* objeto dentro del área */ }
void OnTriggerExit2D(Collider2D other)    { /* objeto salió del área */ }

// Ejemplo: zona de daño
void OnTriggerStay2D(Collider2D other)
{
    if (other.CompareTag("Player"))
        other.GetComponent<PlayerHealth>().TakeDamage(damagePerSecond *
Time.fixedDeltaTime);
}
```

Capas de colisión (Collision Layers)

Las Layers permiten controlar qué objetos colisionan entre sí mediante la Collision Matrix (Edit → Project Settings → Physics 2D). Por ejemplo: el jugador colisiona con enemigos y el terreno, pero las balas del jugador no colisionan con el propio jugador.

- **Capas recomendadas para un juego 2D:** Default, Player, Enemy, Projectile, Ground, Interactable, UI.
- **LayerMask en código:** Se usa para filtrar Raycasts y OverlapCircle a capas específicas, evitando detecciones no deseadas.

Joints

Los Joints conectan dos Rigidbodies con restricciones físicas. Son útiles para cadenas, cuerdas, puentes colgantes, puertas con bisagra y vehículos articulados.

- **HingeJoint2D:** Articulación rotacional alrededor de un punto. Ideal para puertas, ruedas y péndulos.
- **SpringJoint2D:** Conexión elástica con fuerza de restitución. Útil para cuerdas con rebote.
- **DistanceJoint2D:** Mantiene dos objetos a una distancia fija. Ideal para cadenas rígidas.
- **SliderJoint2D:** Permite deslizamiento en un eje. Útil para pistones y plataformas deslizantes.

Raycasting

El Raycasting lanza un rayo invisible desde un punto en una dirección y detecta el primer Collider que intersecta. Es fundamental para detección de suelo, línea de visión, disparos y sistemas de interacción.

```
// Raycasting 2D — detección de suelo
bool IsGrounded()
{
    RaycastHit2D hit = Physics2D.Raycast(
        transform.position,          // origen
        Vector2.down,                // dirección
        groundCheckDistance,         // distancia máxima
        groundLayer                  // LayerMask — solo detecta capa Ground
    );
    return hit.collider != null;
}

// CircleCast — más robusto que Raycast para detección de suelo ancho
bool IsGroundedCircle()
{
    RaycastHit2D hit = Physics2D.CircleCast(
        feetPosition.position, 0.2f, Vector2.down, 0.05f, groundLayer
    );
    return hit.collider != null;
}

// OverlapCircle — detectar todos los colliders en un radio (sin rayo)
Collider2D[] enemies = Physics2D.OverlapCircleAll(
    transform.position, explosionRadius, enemyLayer
);

// Visualizar raycast en editor (solo Debug)
Debug.DrawRay(transform.position, Vector2.down * groundCheckDistance, Color.red);
```

3.2 Animación: Animator Controller, estados, transiciones, Blend Trees, Animation Events e IK básico

> **PUENTE CON UNIDADES ANTERIORES**
> La FSM manual implementada en U2.2 (Idle, Patrol, Chase, Attack) es conceptualmente idéntica al Animator Controller de Unity: ambos son máquinas de estados finitos. La diferencia es que el Animator Controller es una FSM visual especializada en reproducción de clips de animación, mientras que la FSM de U2.2 controla lógica de comportamiento. En U4.2 se verá cómo ambas FSMs pueden coexistir: la del Animator gestiona la animación del NPC mientras la FSM de código gestiona su IA.

El sistema de animación de Unity (Mecanim) permite crear comportamientos de animación complejos mediante un grafo visual de estados y transiciones. Es altamente flexible y soporta animaciones 2D (sprites) y 3D (esqueletos).

Animator Controller — estructura básica

El Animator Controller es el asset (.controller) que define los estados de animación, las transiciones entre ellos y los parámetros que controlan esas transiciones. Se adjunta al componente Animator de un GameObject.

Elemento	Descripción
State (estado)	Nodo que representa un clip de animación o un Blend Tree. Ej: Idle, Run, Jump, Fall.
Transition (flecha)	Conexión entre dos estados con condiciones. Puede ser instantánea o con duración de mezcla (crossfade).
Parameter	Variable que controla las transiciones. Tipos: Float, Int, Bool, Trigger.
Entry / Exit / Any	Nodos especiales: Entry = estado inicial, Any State = transición desde cualquier estado.
Layer	Capas independientes del Animator (ej: cuerpo superior e inferior por separado).
Avatar Mask	Define qué huesos del esqueleto afecta una capa (ej: solo brazos para animación de disparo).

Control del Animator desde código

```
private Animator anim;

// Cachear hashes para evitar búsquedas por string en Update (más eficiente)
static readonly int HashSpeed   = Animator.StringToHash("Speed");
static readonly int HashJump    = Animator.StringToHash("Jump");
static readonly int HashGrounded= Animator.StringToHash("IsGrounded");
static readonly int HashDie     = Animator.StringToHash("Die");

void Awake() => anim = GetComponent<Animator>();

void Update()
{
    // Float: velocidad para Blend Tree de movimiento
    anim.SetFloat(HashSpeed, rb.linearVelocity.magnitude);

    // Bool: estado de contacto con el suelo
    anim.SetBool(HashGrounded, isGrounded);

    // Trigger: eventos puntuales (salto, ataque, muerte)
    if (Input.GetButtonDown("Jump")) anim.SetTrigger(HashJump);
}

public void Die()
{
    anim.SetTrigger(HashDie);
    // La transición a Death state ocurre automáticamente en el Animator
}
```

Blend Trees

Un Blend Tree mezcla múltiples clips de animación de forma fluida basándose en uno o dos parámetros float. Es ideal para animaciones de movimiento donde la transición entre caminar y correr debe ser suave.

- **1D Blend Tree:** Un parámetro controla la mezcla. Ej: Speed controla la transición de Idle (0) → Walk (0.5) → Run (1.0).
- **2D Simple Directional:** Dos parámetros (Horizontal, Vertical) controlan animaciones direccionales. Ideal para movimiento top-down 8 direcciones.
- **2D Freeform Cartesian:** Similar al anterior pero permite distribución libre de poses en el espacio 2D del blend.

Animation Events

Los Animation Events son marcadores en la línea de tiempo de un clip que llaman a un método del script adjunto al mismo GameObject. Son fundamentales para sincronizar efectos con animaciones: sonidos de pasos, partículas de impacto, detección de hitbox en ataques.

```
// Este método es llamado por un Animation Event en el clip de ataque
// El event se agrega en el Animation Window sobre el frame de impacto
public void OnAttackHit()
{
    // Activar hitbox de ataque solo durante el frame de impacto
    attackHitbox.SetActive(true);
    AudioManager.Instance.PlaySFX("sword_swing");
    ParticleSystem slashVFX = ObjectPool.Instance.Get("SlashEffect");
    slashVFX.transform.position = attackPoint.position;
    slashVFX.Play();
}

public void OnAttackEnd()
{
    attackHitbox.SetActive(false);
}

// Para pasos: llamado en cada frame de contacto del pie con el suelo
public void OnFootstep()
{
    AudioManager.Instance.PlaySFX("footstep_grass");
}
```

IK básico (Inverse Kinematics)

La IK permite que las extremidades de un personaje se adapten al entorno de forma procedural, sin necesidad de animaciones específicas para cada situación. Unity incluye IK básica via el método OnAnimatorIK() que se llama en cada frame de animación.

```
// IK para que el personaje mire hacia un objetivo (Look At IK)
void OnAnimatorIK(int layerIndex)
{
    if (lookAtTarget == null) return;

    // Peso de la cabeza mirando al objetivo (0 = sin efecto, 1 = completo)
    anim.SetLookAtWeight(
        weight:     1f,    // peso global
        bodyWeight: 0.2f,  // cuánto rota el cuerpo
        headWeight: 0.7f,  // cuánto rota la cabeza
        eyesWeight: 0.5f,  // cuánto rotan los ojos
        clampWeight: 0.5f  // límite de rotación
```

```
        );
        anim.SetLookAtPosition(lookAtTarget.position);

        // IK de mano derecha sobre un objeto (ej: agarrar una palanca)
        anim.SetIKPositionWeight(AvatarIKGoal.RightHand, 1f);
        anim.SetIKRotationWeight(AvatarIKGoal.RightHand, 1f);
        anim.SetIKPosition(AvatarIKGoal.RightHand, grabTarget.position);
        anim.SetIKRotation(AvatarIKGoal.RightHand, grabTarget.rotation);
    }
```

3.3 Sistema de partículas (VFX Graph / Particle System): efectos visuales para impactos, explosiones y clima

Unity ofrece dos sistemas de partículas: el Particle System clásico (CPU-based, compatible con todas las plataformas) y el VFX Graph (GPU-based, millones de partículas, requiere URP/HDRP). Para la mayoría de juegos 2D y 3D de alcance medio, el Particle System es suficiente y más sencillo de configurar.

Particle System — módulos principales

Módulo	Controla	Uso típico
Main	Duración, loop, velocidad inicial, tamaño, color	Configuración base de todo efecto
Emission	Tasa de emisión y bursts (ráfagas puntuales)	Explosión: burst de 50 partículas en t=0
Shape	Forma del volumen de emisión (esfera, cono, malla)	Lluvia: cono ancho apuntando hacia abajo
Velocity over Lifetime	Velocidad que cambia con el tiempo	Humo: subir lentamente y expandirse
Color over Lifetime	Gradiente de color durante la vida	Fuego: amarillo → naranja → rojo → transparente
Size over Lifetime	Escala que cambia con el tiempo	Burbuja: crece y desaparece
Renderer	Material, modo de renderizado, orden de capas	Sprite o Billboard para 2D
Collision	Colisión con el mundo físico	Gotas de lluvia que rebotan en el suelo
Sub Emitters	Emitir partículas hijas al nacer/morir	Explosión que genera chispas secundarias
Trails	Estela detrás de cada partícula	Meteoros, balas trazadoras

Control del Particle System desde código

```
public class VFXManager : MonoBehaviour
{
    [SerializeField] private ParticleSystem explosionPrefab;
    [SerializeField] private ParticleSystem bloodSplashPrefab;

    // Reproducir efecto en una posición (usando Object Pool para evitar GC)
    public void PlayExplosion(Vector3 position)
    {
```

```
        ParticleSystem ps = Instantiate(explosionPrefab, position,
Quaternion.identity);
        ps.Play();
        Destroy(ps.gameObject, ps.main.duration +
ps.main.startLifetime.constantMax);
    }

    // Cambiar color de partículas en runtime
    public void SetParticleColor(ParticleSystem ps, Color color)
    {
        var main = ps.main;
        main.startColor = color;
    }

    // Emitir un burst puntual sin cambiar la emisión continua
    public void EmitBurst(ParticleSystem ps, int count)
    {
        ps.Emit(count);
    }
}
```

VFX Graph (GPU Particles)

El VFX Graph usa la GPU para simular millones de partículas con lógica de nodos visuales. Requiere Universal Render Pipeline (URP) o HDRP y una GPU que soporte Compute Shaders. Es ideal para efectos masivos como tormentas de partículas, niebla densa, energía mágica y efectos de destrucción a gran escala.

- **Ventajas sobre Particle System:** 10x–100x más partículas simultáneas, lógica visual en nodos, integración con shaders custom, soporte nativo de físicas GPU.
- **Limitaciones:** No compatible con Built-in Render Pipeline, curva de aprendizaje mayor, no disponible en plataformas sin Compute Shaders (algunos móviles).

> **RECOMENDACIÓN**
> Para el Prototipo 2 (plataformero), usar Particle System clásico. VFX Graph es más apropiado para proyectos finales con URP donde se requiere un alto conteo de partículas o efectos especiales complejos.

3.4 Audio: AudioSource, AudioMixer, música adaptativa, efectos de sonido espacial y diseño sonoro básico

El audio es uno de los elementos más impactantes en la experiencia de juego y uno de los más subestimados durante el desarrollo. Unity ofrece un sistema completo que incluye reproducción básica, mezcla de canales, efectos DSP y espacialización 3D.

Componentes principales del sistema de audio

- **AudioClip:** Asset que contiene el audio crudo (WAV, MP3, OGG). Se importa al proyecto y se referencia desde AudioSource.
- **AudioSource:** Componente que reproduce AudioClips. Controla volumen, pitch, loop, espacialización y canal de mezcla.
- **AudioListener:** Componente que actúa como los 'oídos' del jugador. Normalmente en la cámara principal. Solo puede haber uno activo.

- **AudioMixer:** Asset que gestiona grupos de canales (Master, Music, SFX, Voice) con efectos DSP, sends y snapshots.

AudioManager — patrón Singleton

```
public class AudioManager : MonoBehaviour
{
    public static AudioManager Instance { get; private set; }

    [Header("Mixer")]
    [SerializeField] private AudioMixer    masterMixer;

    [Header("Sources")]
    [SerializeField] private AudioSource   musicSource;
    [SerializeField] private AudioSource   sfxSource;

    [Header("Clips")]
    [SerializeField] private AudioClip[]   sfxClips;
    private Dictionary<string, AudioClip>  sfxDict = new();

    void Awake()
    {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject);
        foreach (var clip in sfxClips) sfxDict[clip.name] = clip;
    }

    public void PlaySFX(string name, float pitch = 1f)
    {
        if (!sfxDict.TryGetValue(name, out AudioClip clip)) return;
        sfxSource.pitch = pitch;
        sfxSource.PlayOneShot(clip);
    }

    public void PlayMusic(AudioClip clip, float fadeDuration = 1f)
    {
        StartCoroutine(CrossFadeMusic(clip, fadeDuration));
    }

    IEnumerator CrossFadeMusic(AudioClip newClip, float duration)
    {
        float startVol = musicSource.volume;
        float t = 0;
        while (t < duration)
        {
            musicSource.volume = Mathf.Lerp(startVol, 0, t / duration);
            t += Time.deltaTime;
            yield return null;
        }
        musicSource.clip = newClip;
        musicSource.Play();
        t = 0;
        while (t < duration)
        {
            musicSource.volume = Mathf.Lerp(0, startVol, t / duration);
            t += Time.deltaTime;
            yield return null;
        }
    }

    // Control de volumen via AudioMixer (en dB: 0 = máximo, -80 = silencio)
```

```
    public void SetMusicVolume(float normalizedValue)  // 0..1
    {
        float dB = normalizedValue > 0.001f ? Mathf.Log10(normalizedValue) * 20f
: -80f;
        masterMixer.SetFloat("MusicVolume", dB);
    }
}
```

Música adaptativa

La música adaptativa cambia en respuesta al estado del juego. El enfoque más común es el sistema de capas (layered music): múltiples AudioSources reproducen en sincronía pistas de la misma canción, activando o silenciando capas según la situación.

```
public class AdaptiveMusicSystem : MonoBehaviour
{
    [SerializeField] private AudioSource baseLayer;      // percusión + bajo
siempre activos
    [SerializeField] private AudioSource combatLayer;    // cuerdas y metales en
combate
    [SerializeField] private AudioSource tensionLayer;   // drones en situación
de peligro

    public void SetState(GameMusicState state)
    {
        switch (state)
        {
            case GameMusicState.Explore:
                StartCoroutine(FadeLayer(combatLayer, 0f));
                StartCoroutine(FadeLayer(tensionLayer, 0f));
                break;
            case GameMusicState.Alert:
                StartCoroutine(FadeLayer(tensionLayer, 0.7f));
                break;
            case GameMusicState.Combat:
                StartCoroutine(FadeLayer(combatLayer, 1f));
                StartCoroutine(FadeLayer(tensionLayer, 0.3f));
                break;
        }
    }

    IEnumerator FadeLayer(AudioSource source, float targetVol, float dur = 1f)
    {
        float start = source.volume, t = 0;
        while (t < dur) { source.volume = Mathf.Lerp(start, targetVol, t/dur); t
+= Time.deltaTime; yield return null; }
        source.volume = targetVol;
    }
}
```

Sonido espacial 3D

El sonido 3D (spatial audio) simula la atenuación por distancia y la dirección del sonido según la posición relativa entre AudioSource y AudioListener. Se configura en el componente AudioSource con las curvas de distancia.

- **Spatial Blend:** 0 = sonido 2D (sin espacialización), 1 = sonido 3D completo. Para SFX de mundo (pasos, explosiones) usar 1. Para música y UI usar 0.
- **Min/Max Distance:** Min Distance = radio en el que el sonido está a volumen máximo. Max Distance = radio máximo de audición.

- **Volume Rolloff:** Logarithmic (realista, cae rápido), Linear (constante), Custom (curva diseñada manualmente).

3.5 Iluminación y post-procesado: lightmapping, URP/HDRP básico, Volume profiles y shaders introductorios

La iluminación y el post-procesado definen la identidad visual de un juego. Unity soporta iluminación en tiempo real, iluminación precalculada (baked) y sistemas híbridos. El pipeline de renderizado (Built-in, URP o HDRP) determina qué herramientas están disponibles.

Universal Render Pipeline (URP)

URP es el pipeline de renderizado recomendado para juegos de alcance medio (móvil, PC gama media, consolas portátiles). Ofrece mejor rendimiento que el Built-in RP, soporte completo de post-procesado, 2D Renderer y compatibilidad con el VFX Graph.

Característica	Built-in RP	URP	HDRP
Plataformas target	Todas	Móvil, PC, Consola	PC high-end, Consola AAA
Post-procesado	Post Processing Stack v2	Volume system integrado	Volume system avanzado
Shader Graph	Limitado	Completo	Completo
VFX Graph	No	Sí	Sí
2D Renderer	Limitado	Completo (Lights 2D)	No
Performance	Medio	Alto	Muy alto costo GPU

Tipos de luces en Unity

- **Directional Light:** Luz global que simula el sol. Afecta toda la escena desde una dirección fija. Siempre presente en escenas exteriores.
- **Point Light:** Emite luz en todas direcciones desde un punto. Ideal para antorchas, explosiones y lámparas.
- **Spot Light:** Cono de luz direccional. Ideal para linternas, faros de coches y focos de escenario.
- **Area Light:** Solo disponible en baked lighting. Superficie emisora de luz suave. Ideal para ventanas y paneles.
- **Emissive Materials:** Materiales que emiten luz propia. En URP se combinan con Bloom para efecto de brillo.

Lightmapping (Baked Lighting)

El lightmapping precalcula la iluminación de objetos estáticos y la almacena en texturas (lightmaps). El resultado es visualmente de alta calidad con costo de GPU casi nulo en runtime. Los objetos deben marcarse como 'Static' para participar en el bake.

```
// Configurar un objeto para recibir lightmaps (Static Batching + Lightmap)
// Se hace desde el Inspector: marcar 'Static' en el GameObject

// Cambiar la intensidad de la luz ambiental en runtime
RenderSettings.ambientIntensity = 0.5f;
RenderSettings.ambientLight = new Color(0.1f, 0.1f, 0.2f);  // azul nocturno
```

```
// Cambiar el skybox en runtime
RenderSettings.skybox = nightSkyboxMaterial;
DynamicGI.UpdateEnvironment();  // actualizar GI indirecta
```

Post-procesado con Volume Profiles (URP)

El sistema de Volumes de URP permite aplicar efectos de post-procesado (color grading, bloom, vignette, depth of field) de forma global o local. Un Volume Global afecta toda la cámara; un Volume Local afecta solo cuando la cámara está dentro de su collider.

Efecto	Descripción	Uso en juegos
Bloom	Resplandor de superficies brillantes	Efectos mágicos, explosiones, neón
Color Grading	Corrección de color global (tono, saturación)	Mood: azul frío (terror), amarillo cálido (aventura)
Vignette	Oscurecer los bordes de la pantalla	Tensión, bajo HP, pantalla de foco
Depth of Field	Desenfoque de objetos fuera de foco	Cinemáticas, énfasis en objetos cercanos
Motion Blur	Desenfoque por movimiento rápido	Velocidad extrema, impactos fuertes
Ambient Occlusion	Sombras en grietas y esquinas	Realismo en entornos 3D
Chromatic Aberration	Separación de colores RGB en bordes	Impactos, distorsión, efectos glitch

Shaders introductorios con Shader Graph

Shader Graph es el editor visual de shaders de Unity (disponible en URP y HDRP). Permite crear materiales personalizados sin escribir código HLSL, conectando nodos que representan operaciones matemáticas sobre los píxeles y vértices.

- **Shaders básicos recomendables para el prototipo:**
  - **Dissolve Shader:** Efecto de disolución con noise texture. Útil para muerte de enemigos y teletransportación.
  - **Outline Shader:** Contorno de color alrededor de sprites. Útil para resaltar ítems y personajes seleccionados.
  - **Hit Flash Shader:** Flash blanco al recibir daño. Se implementa con un parámetro float controlado desde código.
  - **Water Shader 2D:** Efecto de ondulación con Vertex displacement y UV scrolling.

3.6 Prototipo 2: Plataformero 2D/3D — físicas, animaciones, VFX, audio y polish visual

El Prototipo 2 es el proyecto integrador de la Unidad 3. Se construye sobre los conceptos de U1 y U2 (mecánicas, patrones, UI, escenas) y añade la capa visual y auditiva completa: físicas precisas, animaciones con Blend Trees, efectos de partículas, audio espacial, post-procesado y una primera sesión de análisis de rendimiento con el Profiler.

Especificación del prototipo

Sistema	Requisito mínimo	Tecnología
Movimiento del jugador	Caminata, carrera, salto con coyote time y jump buffer	Rigidbody2D + FixedUpdate
Animaciones del jugador	Idle, Walk, Run, Jump, Fall, Land con Blend Tree 1D	Animator + Blend Tree
Animaciones de enemigos	Patrol, Alert, Attack con transiciones suaves	Animator + Triggers
VFX de salto/aterrizaje	Polvo al aterrizar, trail en movimiento rápido	Particle System + Object Pool
VFX de combate	Slash al atacar, sangre/impacto al golpear enemigo	Particle System + Animation Event
Audio SFX	Pasos, salto, aterrizaje, ataque, daño, muerte	AudioManager + AudioSource 3D
Música adaptativa	Tranquila en exploración, intensa en combate	AdaptiveMusicSystem con capas
Post-procesado	Bloom, Color Grading y Vignette al bajo HP	URP Volume Profile
Iluminación 2D	Luces 2D con sombras (Sprite Renderer con Normal Maps)	URP 2D Renderer + Point Light 2D
Profiler session	Medir CPU, memoria y draw calls durante gameplay	Unity Profiler

Coyote Time y Jump Buffer

Dos técnicas de game feel esenciales para que el salto del plataformero se sienta responsivo y justo:

```
public class PlatformerController : MonoBehaviour
{
    [SerializeField] private float coyoteTime   = 0.15f; // segundos tras salir
de plataforma
    [SerializeField] private float jumpBuffer   = 0.1f;  // segundos de
anticipación de salto

    private float coyoteTimer;
    private float jumpBufferTimer;

    void Update()
    {
        // Coyote time: mantener ventana de salto tras caer de un borde
        if (isGrounded)  coyoteTimer = coyoteTime;
        else             coyoteTimer -= Time.deltaTime;

        // Jump buffer: registrar salto aunque el jugador no esté en el suelo aún
        if (jumpInput)   jumpBufferTimer = jumpBuffer;
        else             jumpBufferTimer -= Time.deltaTime;

        // Saltar si hay buffer activo Y (está en suelo O dentro de coyote time)
        if (jumpBufferTimer > 0 && coyoteTimer > 0)
        {
            Jump();
            jumpBufferTimer = 0;
            coyoteTimer     = 0;
```

```
        }

        // Salto variable: soltar el botón antes corta el salto
        if (!jumpHeld && rb.linearVelocity.y > 0)
            rb.linearVelocity += Vector2.down * (fallMultiplier - 1) *
Physics2D.gravity.y * Time.deltaTime;
    }
}
```

Primera sesión con el Profiler — ajuste introducido en U3

> **AJUSTE DE SECUENCIA**
> El Profiler se enseña formalmente en U5.5 como parte de la preparación para publicación. Sin embargo, conviene introducirlo aquí porque el Prototipo 2 ya incluye Rigidbody, Particle Systems y AudioSources — suficiente carga para generar datos de rendimiento significativos y detectar problemas tempranos.

El Profiler de Unity (Window → Analysis → Profiler) permite medir en tiempo real el costo de CPU, GPU, memoria, audio y física. En esta sesión introductoria el objetivo es identificar los tres picos más comunes en un plataformero:

- **CPU — Physics:** Demasiados Rigidbodies activos o Colliders complejos. Solución: reducir Fixed Timestep o simplificar Colliders.
- **CPU — Rendering:** Exceso de Draw Calls por materiales no batched. Solución: activar Static Batching en objetos estáticos y usar Sprite Atlas para 2D.
- **Memory — GC Alloc:** Llamadas a Instantiate/Destroy en Update o uso de strings en Update. Solución: Object Pool y caché de hashes de Animator.

```
// Pasos básicos para una sesión de profiling:
// 1. Window → Analysis → Profiler
// 2. Presionar Play en el Editor
// 3. Observar el gráfico de CPU — identificar picos (spikes)
// 4. Hacer clic en un frame con spike → ver la jerarquía de llamadas
// 5. Buscar GC.Alloc en la jerarquía — indica asignaciones de memoria en runtime

// Herramientas adicionales:
// - Frame Debugger (Window → Analysis → Frame Debugger): ver draw calls
// - Memory Profiler (Package Manager → Memory Profiler): snapshot de memoria
// - Physics Debugger (Window → Analysis → Physics Debugger): visualizar
colliders
```

Estructura de scripts del Prototipo 2

```
Assets/_Project/Scripts/
├── Player/
│   ├── PlatformerController.cs  // Movimiento, coyote time, jump buffer
│   ├── PlayerAnimator.cs        // Bridge entre lógica y Animator
(SetFloat/Trigger)
│   ├── PlayerHealth.cs          // HP, invencibilidad temporal (iframes), muerte
│   └── PlayerVFX.cs             // Partículas de pasos, salto, aterrizaje y daño
├── Enemies/
│   ├── PatrolEnemy.cs           // Patrulla con Raycast de borde + Animator
│   └── EnemyAnimator.cs         // Estados de animación del enemigo
├── Environment/
│   ├── MovingPlatform.cs        // Plataforma móvil con Kinematic Rigidbody
```

```
│   ├── BouncePad.cs            // Plataforma de rebote con AddForce Impulse
│   └── Checkpoint.cs          // Trigger que guarda posición de respawn
├── Audio/
│   ├── AudioManager.cs         // Singleton con SFX dict y CrossFade
│   └── AdaptiveMusicSystem.cs  // Capas de música por estado de juego
└── Visual/
    ├── CameraShake.cs          // Cinemachine Impulse para shake en impactos
    └── PostProcessController.cs // Cambiar Volume weight según HP del jugador
```

CONSEJO — GAME FEEL (JUICE)

El 'juice' es el conjunto de micro-reacciones visuales y sonoras que hacen que un juego se sienta satisfactorio: screen shake al aterrizar, freeze frame de 2-3 frames al impactar un enemigo, escalado de sprite (squash & stretch) al saltar, partículas de polvo al correr, sonido de impacto con pitch aleatorio. Ninguna de estas técnicas cambia la mecánica, pero multiplican la sensación de calidad percibida. El Prototipo 2 es el momento ideal para incorporar al menos 3 de estas técnicas.